

EECE 7205: Computing Fundamentals for Engineers

Fall 2026

Lab Assignment #01 Algorithm Analysis Workflow

Topic: Algorithm Analysis

Student Name	Eduardo Scopelli
NUID	003972121
Email	scopellirivas.e@northeastern.edu
Instructor	Dr. Fatema Nafa
Due Date	09-20-2026

Checklist Item

Goal: For each problem, complete the following steps in order. Your report and C++ program must show that you followed the same process used in class.

- **Understand the problem:** Identify the goal, inputs, outputs, constraints, assumptions, and important edge cases.
- **Design the solution:** Select an appropriate algorithm or data structure and briefly explain why it fits the problem.
- **Write pseudocode:** Write clear, language-independent pseudocode before writing the C++ implementation. Use the same pseudocode format and level of detail demonstrated in class.
- **Explain correctness:** Briefly explain how the algorithm produces the required result and handles duplicates, ties, empty inputs, and other relevant edge cases.
- **Analyze complexity:** Determine and justify the time complexity of your algorithm using Big-O notation. Include best-case, average-case, and worst-case complexity when they differ. Also state the space complexity.
- **Implement in C++:** Translate the pseudocode into a clear, functional C++ solution. Use meaningful names, functions, consistent formatting, and comments for non-obvious logic.
- **Test the solution:** Use the provided example and at least two additional test cases for each problem, including at least one edge case. Include screenshots that clearly show the selected problem, input, and output.

Problem 1

Description: Intervals appear in calendars, resource reservations, timelines, and scheduling systems. Overlapping intervals can be combined into a smaller set of non-overlapping intervals.

Question: Given an array of intervals [start, end], sort the intervals by starting value and merge every overlapping interval. Return the resulting non-overlapping intervals.

Understand the problem

- **Goal:** Sort array by start intervals, then merge based on overlapping end intervals.
- **Inputs:** Array of Intervals [start, end]
- **Outputs:** Array of sorted, non-overlapping Intervals [start, end]
- **Constraints:** end > start
- **Assumptions:** Interval values represent hours, touching intervals (eg. [1,2],[2,3]) also get merged, start ≥ 0 , end ≤ 24 , .
- **Edge cases:** Less than 2 intervals: no merge required.

Design the solution

The problem provides a 2D array as an input and requires the same data structure for the output, so no changes in the data structure are required.

As for the logic, the array of intervals must first be sorted by the start times of each interval. This can be accomplished through a simple bubble sort based only on the start values (ignoring overlap and duplicates) which would iterate over every interval, compare its start time to the next interval, and swap them in case the previous interval starts after the next. With the array sorted, overlapping intervals must be merged, considering two main cases: Fully nested intervals and partially overlapping intervals. In the case of fully nested intervals, only the larger interval is maintained and the smaller one is eliminated. In the case of partially overlapping intervals, the smallest start time and largest end time create the new merged interval. In order to be able to iterate over all original intervals, the iterative loop should start on the second element of the array and compare it with the previous element, ensuring that only the previous iteration is affected / deleted without affecting the order of iterations.

Write pseudocode

```
1 n ← SizeOf(Intervals)
2
3 if n <← 1 then
4     return Intervals
5 end if
6
7 for Interval ← 0 to n - 2 do
8     for i ← 0 to n - Interval - 2 do
```

```

9         if Intervals[i].start > Intervals[i + 1].start then
10             swap(Intervals[i], Intervals[i + 1])
11         end if
12     end for
13 end for
14
15 merged ← []
16 prevI ← Intervals[0]
17 for i ← 1 to n - 1 do
18     if Intervals[i].start <← prevI.end then
19         if Intervals[i].end > prevI.end then
20             prevI.end ← Intervals[i].end
21         end if
22     else
23         append prevI to merged
24         prevI ← Intervals[i]
25     end if
26 end for
27 return merged

```

Analyze complexity

Operation	Frequency
$n \leftarrow \text{SizeOf}(\text{Intervals})$	1
Initialize Interval $\leftarrow 0$	1
Check Interval $< n - 2$	$n - 1$
Increment Interval	$n - 2$
Initialize $i \leftarrow 0$	$n - 2$
Check $i < n - \text{Interval} - 2$	$\frac{(n-1)n}{2}$
Increment i	$\frac{(n-2)(n-1)}{2}$
Check $\text{Intervals}[i].\text{start} > \text{Intervals}[i + 1].\text{start}$	$\frac{(n-2)(n-1)}{2}$
Swap $\text{Intervals}[i].\text{start}$ and $\text{Intervals}[i + 1].\text{start}$	$\frac{(n-2)(n-1)}{2}$
$\text{merged} \leftarrow []$	1
$\text{prevI} \leftarrow \text{Intervals}[0]$	1
Initialize $i \leftarrow 1$	1
Check $i < n - \text{Interval} - 2$	$n - 1$
Increment i	$n - 2$
Check $\text{Intervals}[i].\text{start} < \text{prevI}.end$	$n - 2$
Check $\text{Intervals}[i].end > \text{prevI}.end$	$n - 2$
$\text{prevI}.end \leftarrow \text{Intervals}[i].end$	$n - 2$
Else statement	0
Append prevI to merged	$n - 2$
$\text{prevI} \leftarrow \text{Intervals}[i]$	$n - 2$
Return merged	1

Time complexity (Best case): $3 \rightarrow O(1)$

Time complexity (Worst case): $2n^2 + 6n - 7 \rightarrow O(n^2)$

```
// P1
#pragma region
vector<vector<int>> case1_1 = {{1,3}, {15,18}, {8,10}, {2,6}};
vector<vector<int>> ans1_1 = {{1,6}, {8,10}, {15,18}};
assert(mergeIntervals(case1_1) == ans1_1);

vector<vector<int>> case1_2 = {{1,7}, {7,10}, {2,4}};
vector<vector<int>> ans1_2 = {{1,10}};
assert(mergeIntervals(case1_2) == ans1_2);

vector<vector<int>> case1_3 = {{1,5}};
vector<vector<int>> ans1_3 = {{1,5}};
assert(mergeIntervals(case1_3) == ans1_3);

cout << "Problem 1 Test Cases: Passed" << endl;
#pragma endregion

# (.venv) scopellie@DESKTOP-0P0W0FH:~/EECE7205$ /home/scopellie/EECE7205/Assignment01_Scopellie
Problem 1 Test Cases: Passed
Problem 2 Test Cases: Passed
Problem 3 Test Cases: Passed
Problem 4 Test Cases: Passed
Problem 5 Test Cases: Passed
Problem 6 Test Cases: Passed
Problem 7 Test Cases: Passed
Problem 8 Test Cases: Passed
Problem 9 Test Cases: Passed
Problem 10 Test Cases: Passed
```

Implement in C++

Problem 2

Description: A manufacturing line labels items with three categories represented by 0, 1, and 2. The items must be grouped in category order.

Question: Given an array containing only 0, 1, and 2, sort it in place. First implement a counting-based solution. Then implement the one-pass Dutch National Flag algorithm without using `std::sort()`.

Understand the problem

- **Goal:** Sort array containing only 0, 1 and 2 categories by a counting method, and by the one-pass Dutch National Flag algorithm.
- **Inputs:** Array containing 0, 1 and 2.
- **Outputs:** Sorted array.
- **Constraints:** Values are either 0, 1 or 2.
- **Assumptions:**
- **Edge cases:**

0.1 Case 1. Count-based solution

Design the solution

The problem provides an array of integers/categories with the need to sort this array, for which the data structure can be maintained.

In order to solve the problem through a counting method, we have to establish a counter per class (0, 1 and 2), iterate over the original array to establish the frequency of each class, and create a new array in which we will push each class in order their corresponding amount of times based on the counters.

Write pseudocode

```
1 n ← SizeOf(Labels)
2 count0 ← 0
3 count1 ← 0
4 count2 ← 0
5 for i ← 0 to n - 1 do
6     if Labels[i] = 0 then
7         count0 ← count0 + 1
8     else if Labels[i] = 1 then
9         count1 ← count1 + 1
10    else
11        count2 ← count2 + 1
12    end if
13 end for
14
15 sorted ← []
16 for i ← 0 to n - 1 do
17     if i < count0 then
18         append 0 to sorted
19     else if i < count0 + count1
20         append 1 to sorted
21     else
22         append 2 to sorted
23     end if
24 end for
25 return sorted
```

Analyze complexity

Operation	Frequency
$n \leftarrow \text{SizeOf}(\text{Labels})$	1
$\text{count0} \leftarrow 0$	1
$\text{count1} \leftarrow 0$	1
$\text{count2} \leftarrow 0$	1
Initialize $i \leftarrow 0$	1
Check $i < n - 1$	n
Increment i	$n - 1$
Check $\text{Labels}[i] == 0$	$n - 1$
Increment count0	$n - 1$
Check $\text{Labels}[i] == 1$	$n - 1$
Increment count1	$n - 1$
Else statement	0
Increment count2	$n - 1$
$\text{sorted} \leftarrow []$	1
Initialize $i \leftarrow 0$	1
Check $i < n - 1$	n
Increment i	$n - 1$
Check $i < \text{count0}$	$n - 1$
Append 0 to sorted	$n - 1$
Check $i < \text{count0} + \text{count1}$	$n - 1$
Append 1 to sorted	$n - 1$
Else statement	0
Append 2 to sorted	$n - 1$
Return merged	1

Time complexity (Best case): $8n + 2 \rightarrow O(n)$

Time complexity (Worst case): $10n \rightarrow O(n)$

Implement in C++

0.2 Case 2. One-pass Dutch National Flag

Design the solution

The problem provides an array of integers/categories with the need to sort said array, for which the data structure can be maintained.

In this case the One-pass Dutch National Flag algorithm will be used to solve the problem.

Write pseudocode

```

1 n ← SizeOf(Labels)
2 low ← 0
3 mid ← 0
4 high ← n - 1
5 for i ← 0 to n - 1 do
6     if Labels[mid] = 0 then

```

```

7       swap(Labels[mid], Labels[low])
8       low ← low + 1
9       mid ← mid + 1
10      else if Labels[mid] = 1
11          mid ← mid + 1
12      else
13          swap(Labels[mid], Labels[high])
14          high ← high - 1
15      end if
16  end for
17  return Labels

```

Explain correctness

—

Analyze complexity

Operation	Frequency
$n \leftarrow \text{SizeOf}(\text{Labels})$	1
$low \leftarrow 0$	1
$mid \leftarrow 0$	1
$high \leftarrow n - 1$	1
Initialize $i \leftarrow 0$	1
Check $i < n - 1$	n
Increment i	$n - 1$
Check $\text{Labels}[mid] == 0$	$n - 1$
Swap $\text{Labels}[mid]$ and $\text{Labels}[low]$	$n - 1$
Increment low	$n - 1$
Increment mid	$n - 1$
Check $\text{Labels}[mid] == 1$	$n - 1$
Increment mid	$n - 1$
Else statement	0
Swap $\text{Labels}[mid]$ and $\text{Labels}[high]$	$n - 1$
Decrement $high$	$n - 1$
Return Labels	1

Time complexity (Best case): $6n + 1 \rightarrow O(n)$

Time complexity (Worst case): $5n + 2 \rightarrow O(n)$

Implement in C++

Problem 3

Description: A custom ordering is sometimes required when the normal numeric order does not produce the desired combined result.

Question: Given a vector of nonnegative integers, arrange them so that their concatenation

forms the largest possible number. Return the answer as a string.

Understand the problem

- **Goal:** Sort array in such a way that its concatenate values into a string.
- **Inputs:** Array of nonnegative integers.
- **Outputs:** String.
- **Constraints:**
- **Assumptions:**
- **Edge cases:**

Design the solution

Hello.

Write pseudocode

```
1 n ← SizeOf(Vector)
2 for v ← 0 to n - 2 do
3     for i ← 0 to n - v - 2 do
4         digits1 ← round up log10(Vector[i])
5         digits2 ← round up log10(Vector[i + 1])
6         num1 ← Vector[i] * 10^digits1 + Vector[i + 1]
7         num2 ← Vector[i + 1] * 10^digits1 + Vector[i]
8         if num1 < num2 then
9             swap(Vector[i], Vector[i + 1])
10        end if
11    end for
12 end for
13 Order ← ""
14 for i ← 0 to n - 1 do
15     append Vector[i] to Order
16 end for
17 return Order
```

Explain correctness

Hello.

Analyze complexity

Operation	Frequency
$n \leftarrow \text{SizeOf}(\text{Vector})$	1
Initialize $v \leftarrow 0$	1
Check $v < n - 2$	$n - 1$
Increment v	$n - 2$
Initialize $i \leftarrow 0$	$n - 2$
Check $i < n - v - 2$	$\frac{(n-1)n}{2}$
Increment i	$\frac{(n-2)(n-1)}{2}$
Compute $\text{digits1} \leftarrow \log_{10}(\text{Vector}[i])$	$\frac{(n-2)(n-1)}{2}$
Round up previous computation	$\frac{(n-2)(n-1)}{2}$
Compute $\text{digits2} \leftarrow \log_{10}(\text{Vector}[i + 1])$	$\frac{(n-2)(n-1)}{2}$
Round up previous computation	$\frac{(n-2)(n-1)}{2}$
Compute $\text{num1} \leftarrow \text{Vector}[i] * 10^{\text{digits1}} + \text{Vector}[i + 1]$	$\frac{(n-2)(n-1)}{2}$
Compute $\text{num2} \leftarrow \text{Vector}[i + 1] * 10^{\text{digits2}} + \text{Vector}[i]$	$\frac{(n-2)(n-1)}{2}$
Check $\text{num1} < \text{num2}$	$\frac{(n-2)(n-1)}{2}$
Swap $\text{Vector}[i]$ and $\text{Vector}[i + 1]$	$\frac{(n-2)(n-1)}{2}$
Order $\leftarrow ""$	1
Initialize $i \leftarrow 0$	1
Check $i < n - 1$	n
Increment i	$n - 1$
Append $\text{Vector}[i]$ to Order	$n - 1$
Return Order	1

Time complexity: $O(n^2)$

Implement in C++

Problem 4

Description: Anagrams contain the same characters with the same frequencies but in a different order. Grouping them is useful in text indexing and search.

Question: Given a vector of strings, group the anagrams together. The order of groups does not matter, but words within each group should retain their original input order.

Understand the problem

- **Goal:** Group anagrams together.
- **Inputs:** Vector of strings.
- **Outputs:** Array of vectors containing grouped anagrams.
- **Constraints:**
- **Assumptions:**

- Edge cases:

Design the solution

Hello.

Write pseudocode

```
1 n ← SizeOf(Words)
2 for v ← 0 to n - 2 do
3     for i ← 0 to n - v - 2 do
4         digits1 ← round down log10(Vector[i]) + 1
5         digits2 ← round down log10(Vector[i + 1]) + 1
6         num1 ← Vector[i] * 10^digits2 + Vector[i + 1]
7         num2 ← Vector[i + 1] * 10^digits1 + Vector[i]
8         if num1 < num2 then
9             swap(Vector[i], Vector[i + 1])
10        end if
11    end for
12 end for
13 Order ← ""
14 for i ← 0 to n - 1 do
15     append Vector[i] to Order
16 end for
17 return Order
```

Explain correctness

Hello.

Analyze complexity

Operation	Frequency
$n \leftarrow \text{SizeOf}(\text{Vector})$	1
Initialize $v \leftarrow 0$	1
Check $v < n - 2$	$n - 1$
Increment v	$n - 2$
Initialize $i \leftarrow 0$	$n - 2$
Check $i < n - v - 2$	$\frac{(n-1)n}{2}$
Increment i	$\frac{(n-2)(n-1)}{2}$
Compute $\text{digits1} \leftarrow \log_{10}(\text{Vector}[i])$	$\frac{(n-2)(n-1)}{2}$
Round up previous computation	$\frac{(n-2)(n-1)}{2}$
Compute $\text{digits2} \leftarrow \log_{10}(\text{Vector}[i + 1])$	$\frac{(n-2)(n-1)}{2}$
Round up previous computation	$\frac{(n-2)(n-1)}{2}$
Compute $\text{num1} \leftarrow \text{Vector}[i] * 10^{\text{digits1}} + \text{Vector}[i + 1]$	$\frac{(n-2)(n-1)}{2}$
Compute $\text{num2} \leftarrow \text{Vector}[i + 1] * 10^{\text{digits2}} + \text{Vector}[i]$	$\frac{(n-2)(n-1)}{2}$
Check $\text{num1} < \text{num2}$	$\frac{(n-2)(n-1)}{2}$
Swap $\text{Vector}[i]$ and $\text{Vector}[i + 1]$	$\frac{(n-2)(n-1)}{2}$
Order $\leftarrow ""$	1
Initialize $i \leftarrow 0$	1
Check $i < n - 1$	n
Increment i	$n - 1$
Append $\text{Vector}[i]$ to Order	$n - 1$
Return Order	1

Time complexity: $O(n^2)$

Implement in C++